LFX Project Proposal

Project Title: Implementation of the RISC-V ISA Extensions for Control-Flow Integrity

Author
Muhammad Waleed Akram

Sargantana Core
Repository:
https://github.com/bsc-loca/sargantana

Contents

Chapter 1

Project Overview

1.1 Introduction

Control-Flow Integrity (CFI) has become an essential hardware security mechanism for protecting modern processors against control-flow hijacking attacks. Software vulnerabilities such as stack-smashing, buffer overflows, and use-after-free errors can allow an attacker to overwrite return addresses or corrupt indirect branch targets, redirecting program execution toward malicious code. These attacks compromise the intended execution path of a program while often remaining undetected by conventional protection mechanisms.

To address these challenges, the RISC-V architecture has recently introduced two standardized ISA extensions that provide hardware-assisted Control-Flow Integrity. The **Zicfiss** (Shadow Stack) extension protects function return addresses by maintaining a separate hardware-protected stack, while the **Zicfilp** (Landing Pad) extension ensures that indirect branches can only transfer control to explicitly marked landing pad locations. Together, these extensions provide complementary protection against a wide range of control-flow attacks.

This project focuses on implementing both of these RISC-V CFI extensions on **Sargantana**, an open-source, Linux-capable, out-of-order RISC-V processor developed by the Barcelona Supercomputing Center (BSC). The implementation requires understanding the existing processor microarchitecture, identifying the pipeline components affected by the new ISA extensions, integrating the required hardware mechanisms, and validating their correctness through simulation and system-level testing.

1.2 Project Objectives

The primary objective of this project is to implement hardware support for the RISC-V Control-Flow Integrity ISA extensions on the Sargantana processor. The implementation will focus on both the **Landing Pad (Zicfilp)** and **Shadow Stack (Zicfiss)** mechanisms while maintaining compatibility with the existing processor pipeline.

The project objectives include:

- Study the RISC-V Control-Flow Integrity ISA specifications.
- Analyze the Sargantana processor microarchitecture and RTL implementation.
- Identify the processor pipeline components requiring modification.
- Develop an architectural integration plan for both CFI extensions.
- Implement hardware support for the Shadow Stack mechanism.

- Implement hardware support for Landing Pad validation.
- Integrate the new functionality into the existing processor pipeline.
- Develop comprehensive RTL verification and testing.
- Evaluate the functional correctness and hardware overhead of the implementation.

1.4 Expected Deliverables

The expected outcome of this project is a complete architectural and RTL implementation of the RISC-V Control-Flow Integrity extensions within the Sargantana processor. The implementation will include the necessary hardware modifications, verification infrastructure, and supporting documentation required for upstream integration.

The major deliverables include:

- Architectural integration plan for CFI support.
- RTL implementation of the Zicfiss (Shadow Stack) extension.
- RTL implementation of the Zicfilp (Landing Pad) extension.
- Processor pipeline modifications required for CFI support.
- Verification environment and regression test suite.
- Functional validation using the Sargantana emulation infrastructure.
- Performance and hardware overhead evaluation.
- Technical documentation describing the implementation and verification process.

1.5 Repository and RTL Study

A comprehensive microarchitecture document was prepared during this phase, covering the complete processor pipeline from instruction fetch to commit, including detailed descriptions of the RTL hierarchy, pipeline stages, execution units, recovery mechanisms, and repository organization. Particular attention was given to identifying the components that are likely to require modifications during the integration of the Control-Flow Integrity extensions.

1.6 Reference Specifications and Technical Blogs

The implementation proposed in this document is based on the official RISC-V Control-Flow Integrity specifications together with an extensive study of the Sargantana processor microarchitecture. To document the learning process and provide structured technical references, a series of technical blogs and supporting documents were prepared covering both the ISA specifications and the processor architecture.

Primary References

- RISC-V Unprivileged ISA Specification — Control-Flow Integrity Extensions
- RISC-V Privileged ISA Specification — Control-Flow Integrity Extensions
- Sargantana Open-Source RISC-V Processor Repository

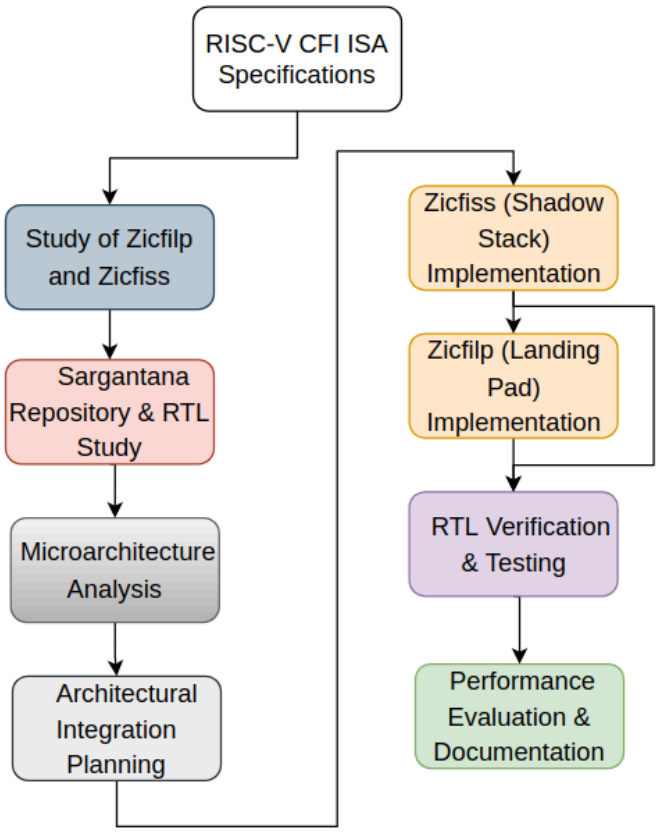
- Sargantana RTL Microarchitecture Study

Blogs

- **Blog 1:** Understanding the RISC-V Control-Flow Integrity (CFI) ISA - Part 1: Landing Pads (Zicfilp)
- **Blog 2:** Understanding the RISC-V Control-Flow Integrity ISA — Part 2 (Shadow Stack / Zicfiss)
- **Blog 3:** Understanding the RISC-V Control-Flow Integrity (CFI) ISA - Part 3: Privileged Architecture
- **Blog 4:** From a Simple FSM to Hardware Control-Flow Integrity: Understanding the Purpose of the RISC-V CFI Coding Challenge

Figure 1.1 Overall Project Workflow

Since this is only the overview chapter, I **wouldn't include a detailed implementation diagram here**. Instead, use a simple workflow that introduces the project:



Chapter 2

Background

2.1 RISC-V CFI ISA Extensions

The RISC-V Control-Flow Integrity architecture consists of two independent yet complementary ISA extensions. Each extension protects a different aspect of program control flow and may be implemented independently or together depending on the processor design.

2.1.1 Zicfilp (Landing Pad)

The **Zicfilp** extension protects indirect control transfers by ensuring that every indirect jump or indirect function call can only branch to a valid landing pad.

Instead of allowing indirect branches to jump to any executable instruction, programs compiled with Zicfilp explicitly mark valid entry points using a dedicated Landing Pad (LPAD) instruction. During execution, whenever an indirect branch is resolved, the processor verifies that the destination begins with a valid landing pad instruction. If the destination is not a valid landing pad, a software-check exception is generated, preventing execution from continuing along an unauthorized path.

2.1.2 Zicfiss (Shadow Stack)

The **Zicfiss** extension protects function returns by maintaining a hardware-protected copy of return addresses known as the **shadow stack**.

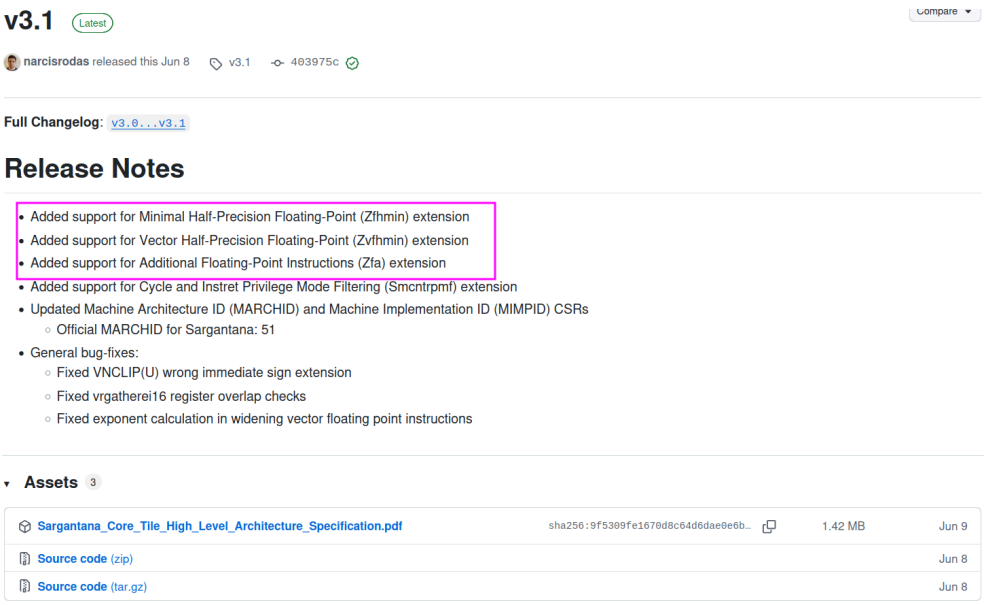
Whenever a function call occurs, the processor stores the return address on both the normal software stack and the protected shadow stack. During a function return, the processor compares the return address from the regular stack with the protected copy stored in the shadow stack. Execution continues only if both values match.

If an attacker overwrites the software stack's return address, the mismatch is detected immediately, preventing execution from returning to an unauthorized location. Since the shadow stack is protected through dedicated hardware semantics and controlled access mechanisms, malicious software cannot modify its contents through ordinary memory operations.

2.2 Existing ISA Extension Integration in Sargantana

One of the notable characteristics of the Sargantana processor is its modular approach to supporting optional RISC-V ISA extensions. Rather than embedding all functionality into a single monolithic design, the repository organizes extension-specific logic into dedicated RTL modules while integrating them through the existing pipeline and execution infrastructure.

The repository already includes support for several standardized extensions, including components related to floating-point arithmetic, vector processing, and floating-point enhancement instructions. Examples include modules implementing functionality associated with extensions such as **Zfa**, **Zvfhmin**, floating-point conversion units, SIMD functional units, and specialized execution resources.



v3.1 Latest

narcisrodas released this Jun 8 · v3.1 · 463975c

Full Changelog: `v3.0...v3.1`

Release Notes

- Added support for Minimal Half-Precision Floating-Point (Zfhmin) extension
- Added support for Vector Half-Precision Floating-Point (Zvfhmin) extension
- Added support for Additional Floating-Point Instructions (Zfa) extension
- Added support for Cycle and Instret Privilege Mode Filtering (Smcntrpmf) extension
- Updated Machine Architecture ID (MARCHID) and Machine Implementation ID (MIMPID) CSRs
  - Official MARCHID for Sargantana: 51
- General bug-fixes:
  - Fixed VNCLIP(U) wrong immediate sign extension
  - Fixed vrgatherei16 register overlap checks
  - Fixed exponent calculation in widening vector floating point instructions

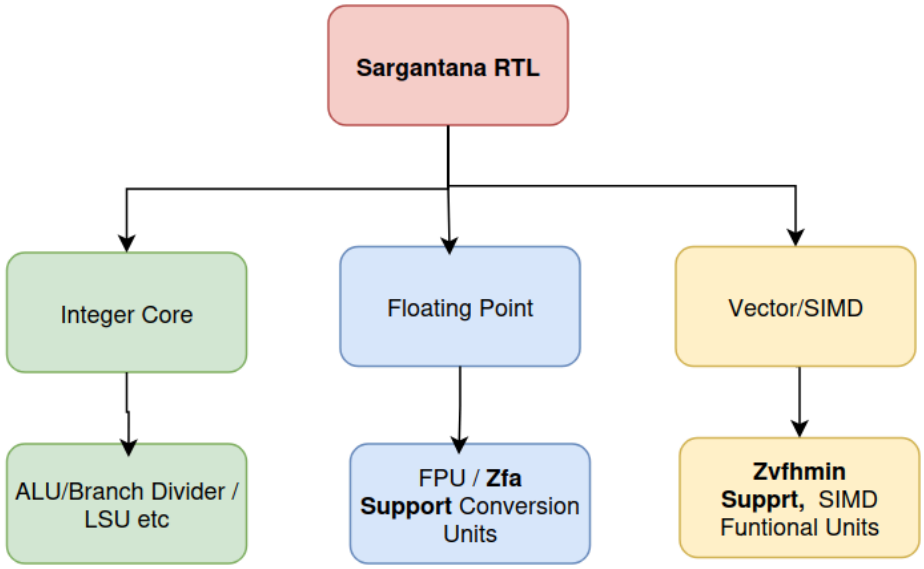
Assets 3

Sargantana_Core_Tile_High_Level_Architecture_Specification.pdf	sha256:9f5309fe1670d8c64d6dae0e6b…	1.42 MB	Jun 9
Source code (zip)			Jun 8
Source code (tar.gz)			Jun 8

This existing organization demonstrates a scalable methodology for extending the processor architecture. New ISA features are integrated by adding extension-specific hardware where required while reusing the existing pipeline, control logic, execution units, and verification infrastructure. This modular design philosophy makes Sargantana well suited for incorporating additional standardized extensions such as the RISC-V Control-Flow Integrity extensions.

Existing Extension Organization

An idea ;



Chapter 3

Overall Integration Strategy

3.1 Integration Philosophy

The implementation strategy aims to integrate the RISC-V Control-Flow Integrity (CFI) extensions into the existing Sargantana processor while preserving its modular pipeline organization and minimizing changes to unrelated components. Rather than redesigning the processor, the objective is to introduce dedicated security mechanisms that operate alongside the existing execution pipeline.

The implementation follows the architecture defined by the official RISC-V CFI specification, where the two ISA extensions provide complementary protection mechanisms.

- **Zicfilp (Landing Pad)** protects indirect control-flow transfers by ensuring they target only explicitly authorized landing locations.
- **Zicfiss (Shadow Stack)** protects function return addresses by maintaining an independent hardware-protected stack.

Both extensions operate independently but together provide comprehensive hardware-assisted control-flow integrity.

The overall philosophy is based on the following principles:

- Preserve existing pipeline functionality.
- Reuse current pipeline control whenever possible.
- Introduce new logic only where required.
- Maintain compatibility with software that does not use CFI.
- Keep verification modular by validating each extension separately before full integration.

3.2 High-Level CFI Architecture

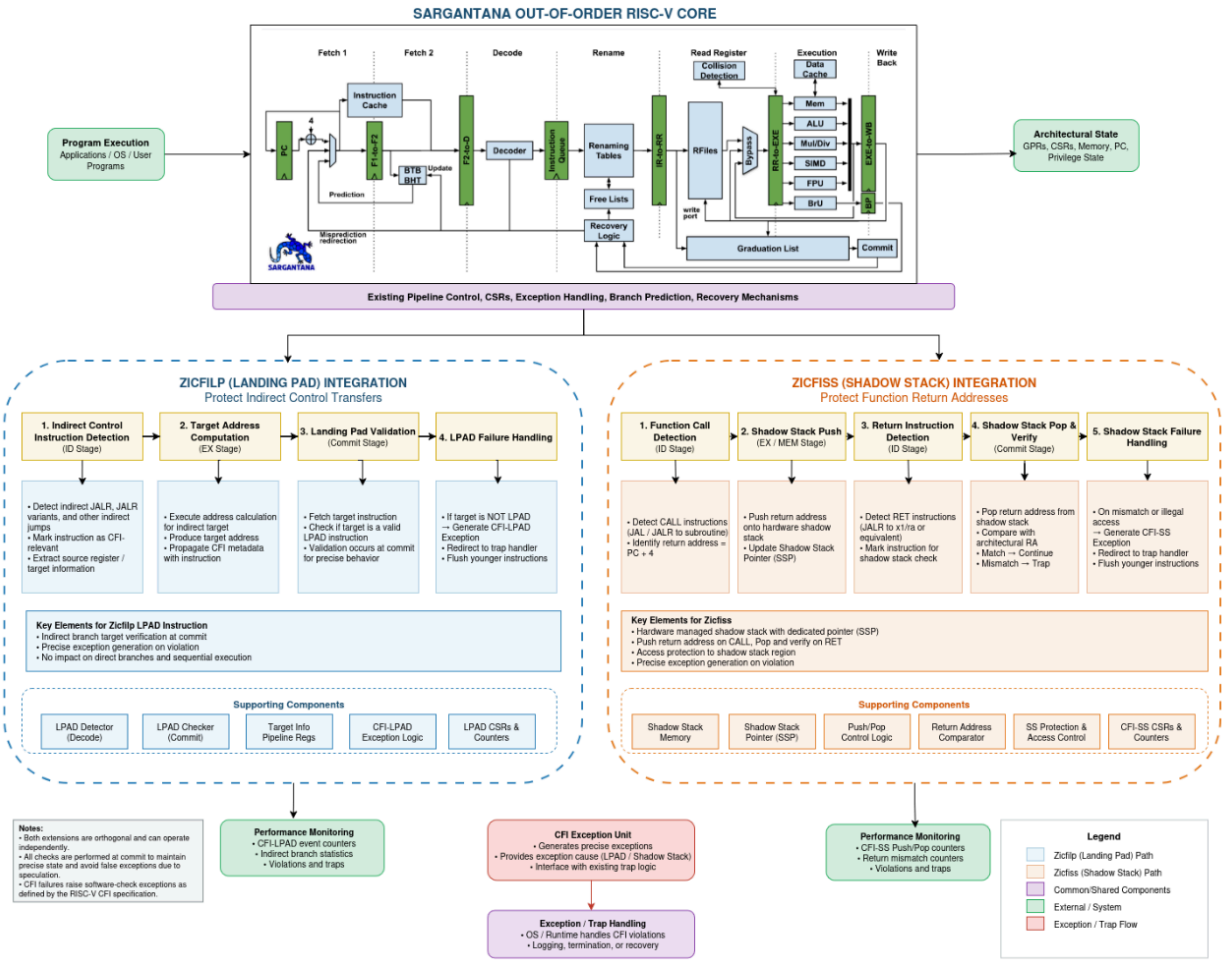
At a high level, the proposed architecture introduces two additional security paths into the processor.

The first path monitors all indirect control-flow instructions. Whenever an indirect jump or call is executed, the processor verifies that the destination begins with a valid Landing Pad (LPAD) instruction before allowing execution to continue.

The second path operates during function calls and returns. Every call stores its return address in both the architectural stack and a hardware-managed shadow stack. During a return instruction, the processor compares the architectural return address against the protected shadow copy. Execution continues only if both addresses match.

These two mechanisms operate in parallel while remaining largely independent of each other. They share the existing instruction fetch, decode, execution, and commit infrastructure without requiring major pipeline redesign.

Conceptually, the processor becomes: (NOTE: I have not shown completeness here (like not all stages; just giving a high level Overview) . Next I will go in detail for both LPAD and SS where I will add a detailed figure .)



3.3 Zicfilp Integration Path

The Landing Pad extension introduces additional validation for indirect control-flow instructions.

The integration path consists of four conceptual stages:

1. Detect indirect control-flow instructions.
2. Carry LPAD-related information through the pipeline.
3. Validate the landing pad at the target address.
4. Raise a software-check exception if validation fails.

Since LPAD validation affects only indirect transfers, direct branches and sequential execution continue to operate without modification.

3.4 Zicfiss Integration Path

The Shadow Stack extension introduces protected storage for return addresses.

During every function call, the return address is written simultaneously to:

- the normal software stack, and
- the hardware-managed shadow stack.

When a return instruction executes, the processor retrieves the return address from both locations and compares them before allowing the return to complete.

If the values differ, execution is immediately redirected to the exception handling mechanism defined by the CFI specification.

The overall Shadow Stack integration consists of four conceptual stages:

1. Detect function call instructions.
2. Push return addresses into the shadow stack.
3. Detect return instructions.
4. Compare architectural and shadow return addresses.

3.5 Overall Pipeline Modifications

The proposed implementation preserves the existing Sargantana pipeline while extending several stages with additional CFI functionality.

The instruction fetch stages continue to fetch instructions normally. The decode stage identifies CFI-related instructions and generates the necessary control information.

The execution stage computes branch targets and return addresses exactly as before, while additional CFI metadata propagates alongside the existing pipeline signals.

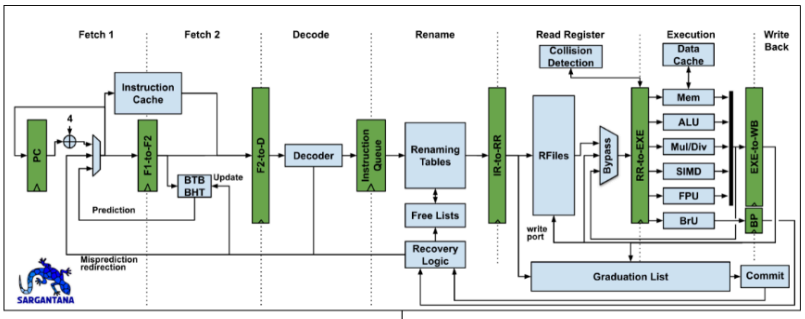
During commit, the processor performs the final security checks:

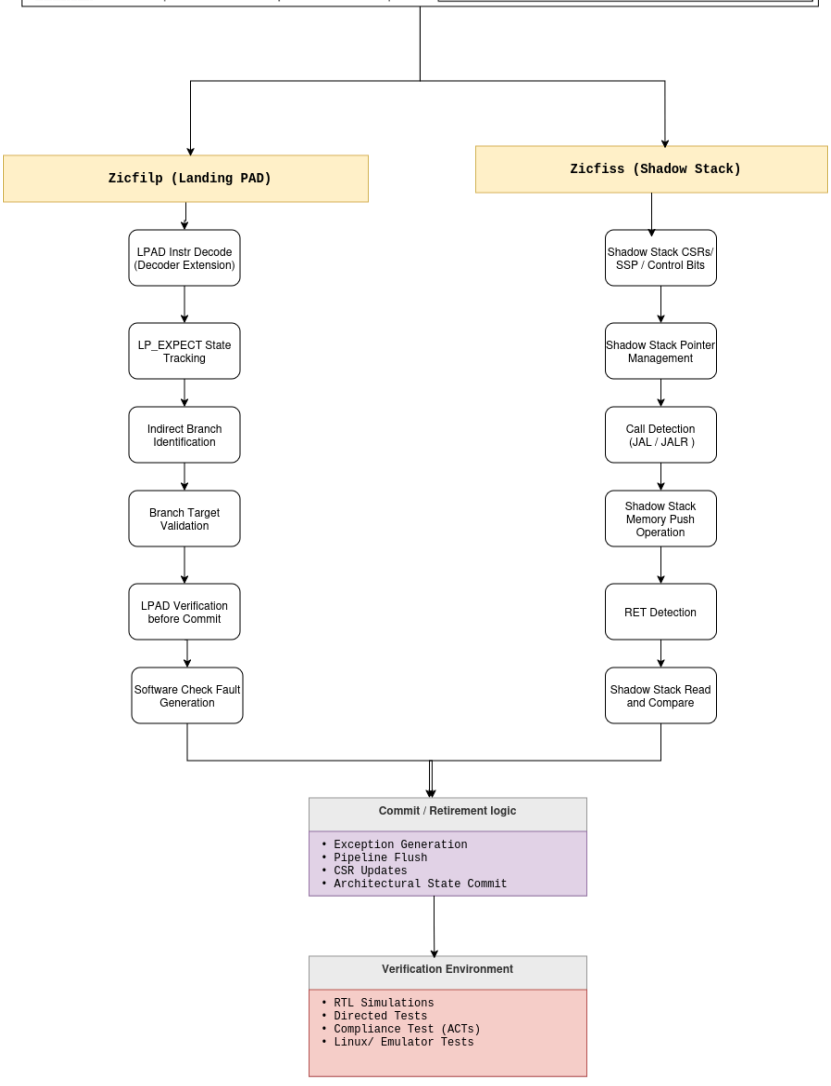
- Landing Pad validation for indirect branches.
- Shadow Stack verification for returns.

Any detected violation results in a software-check exception, allowing the processor to maintain precise exception behavior consistent with the RISC-V specification.

Complete details are in Chapters 6 and 9.

FlowChart


Fetch 1
Fetch 2
Decode
Rename
Read Register
Execution
Write Back
Collision Detection
Data Cache
Instruction Cache
PC
4
F1-to-F2
BTB
BHT
Update
Prediction
F2-to-D
Decoder
Instruction Queue
Renaming Tables
Free Lists
Recovery Logic
IR-to-RR
RFiles
write port
Bypass
RR-to-EXE
Mem
ALU
Mul/Div
SIMD
FPU
BrU
EXE-to-WB
LBP
Misprediction redirection
Graduation List
Commit
SARGANTANA


Zicfilp (Landing PAD)
Zicfiss (Shadow Stack)
LPAD Instr Decode (Decoder Extension)
LP_EXPECT State Tracking
Indirect Branch Identification
Branch Target Validation
LPAD Verification before Commit
Software Check Fault Generation
Shadow Stack CSRs/ SSP / Control Bits
Shadow Stack Pointer Management
Call Detection (JAL / JALR)
Shadow Stack Memory Push Operation
RET Detection
Shadow Stack Read and Compare
Commit / Retirement logic
• Exception Generation
• Pipeline Flush
• CSR Updates
• Architectural State Commit
Verification Environment
• RTL Simulations
• Directed Tests
• Compliance Test (ACTs)
• Linux/ Emulator Tests

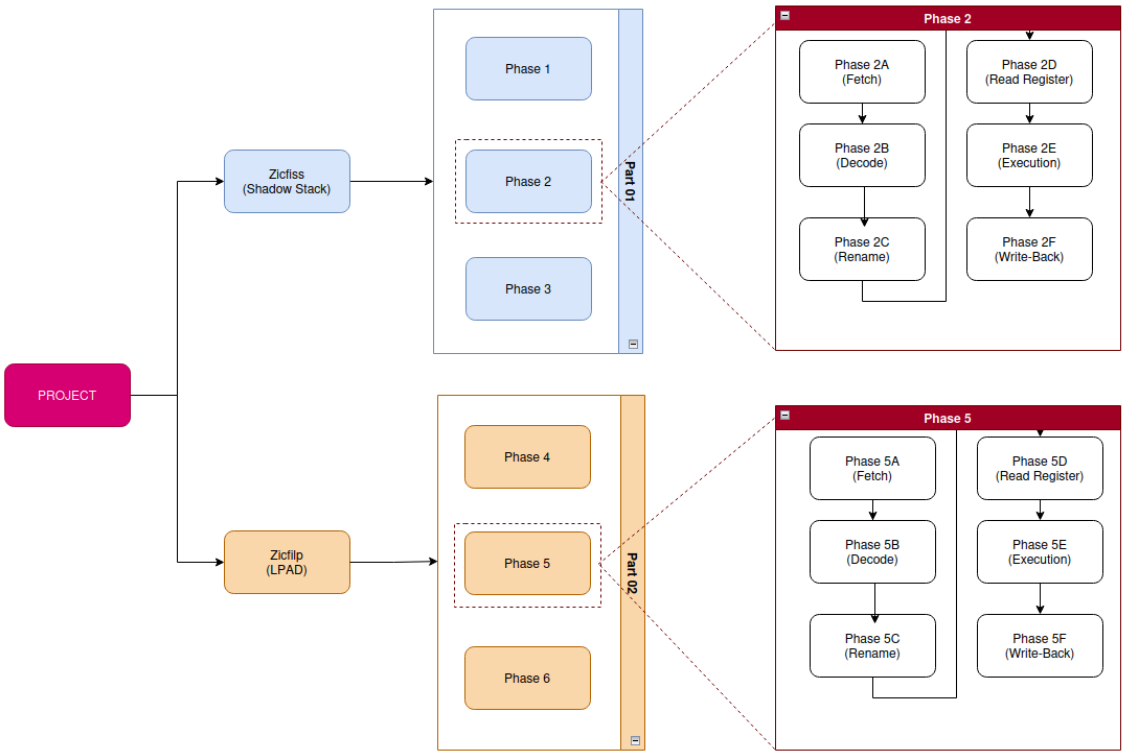
Chapter 4

Phased Implementation Plan

4.1 Overview

To ensure a structured and incremental development process, the implementation is divided into two major parts corresponding to the two RISC-V Control-Flow Integrity ISA extensions. Each part consists of 3 phases that gradually progress from architectural integration to complete functional verification. This phased methodology minimizes development risk, enables continuous validation, and allows each extension to be completed and tested independently before integrating the next.

The first half of the project focuses on implementing the **Zicfiss (Shadow Stack)** extension, while the second half implements the **Zicfilp (Landing Pad)** extension. Within each part, the first two phases are dedicated to hardware implementation and pipeline integration, whereas the third phase focuses entirely on verification, debugging, compliance testing, and performance evaluation.



4.2 Part I – Zicfiss (Shadow Stack)

The first implementation stage focuses on introducing hardware-enforced shadow stack protection into the Sargantana processor. This includes architectural support for protected return addresses, integration with the processor pipeline, enforcement of shadow stack semantics, and comprehensive verification of the implemented functionality.

The implementation is divided into the following phases:

Phase	Description
Phase 1	Zicfiss foundation and architectural preparation
Phase 2	Zicfiss Pipelined Integration
Phase 3	Zicfiss **verification**

4.3 Part II – Zicfilp (Landing Pad)

The second implementation stage introduces hardware support for the Landing Pad extension, enabling validation of indirect control-flow transfers before architectural commitment. This stage integrates LPAD instruction support, landing pad verification logic, software-check exception generation, and complete validation of indirect branch protection.

The implementation is divided into the following phases:

Phase	Description
Phase 4	Zicfilp foundation and architectural preparation
Phase 5	Zicfilp Pipelined Integration
Phase 6	Zicfilp **verification**

4.4 Chapter Organization

The remainder of this proposal follows the implementation roadmap presented above. Chapters **5 through 7** describe the detailed implementation plan for the three Zicfiss phases, including the architectural modifications, pipeline integration, RTL implementation strategy, and verification methodology. Chapters **8 through 10** then present the corresponding implementation plan for the Zicfilp extension using the same phased approach. Finally, the proposal concludes with the overall project timeline, expected milestones, and deliverables.

Chapter 5

Phase 1 – Zicfiss Foundation and Architectural Preparation

5.1 Objectives

The first phase establishes the architectural foundation required to support the RISC-V Zicfiss (Shadow Stack) extension within the Sargantana processor. Before modifying any pipeline stage, it is necessary to identify the new architectural resources, hardware structures, and processor state that must be introduced to enable protected return-address management.

Unlike the subsequent implementation phases, this phase does not focus on pipeline integration or RTL development. Instead, it defines the fundamental building blocks that will later be connected to the processor pipeline. The outcome of this phase is a complete architectural framework upon which the remaining implementation can be developed.

5.2 Existing Return Path Analysis

The first step is to understand how function calls and returns are currently handled by the processor. In a conventional RISC-V processor, the return address generated during a function call is stored only within the normal software stack. Since this stack resides in ordinary memory, it can potentially be modified through memory corruption vulnerabilities such as stack overflows or arbitrary memory writes.

When a return instruction is executed, the processor simply loads the stored return address and transfers control to the corresponding target without independently verifying whether the address has been altered. Consequently, the integrity of the control flow depends entirely on the correctness of software-managed memory.

5.3 Architectural State Requirements

Supporting the Shadow Stack requires the processor to maintain additional architectural state beyond the standard RISC-V execution context. The most important addition is the Shadow Stack Pointer (SSP), which tracks the current position within the protected Shadow Stack.

Along with the SSP, the processor must maintain control information describing whether the Shadow Stack mechanism is enabled and whether current execution is permitted to access or modify the protected state. Additional architectural information is also required to correctly preserve Shadow Stack operation during privilege transitions, interrupts, and exception handling.

5.4 Pipeline Preparation

Although the detailed pipeline integration is deferred to the next phase, several architectural considerations must be established before implementation begins. The processor pipeline must eventually be capable of identifying function call and return instructions, transporting Shadow Stack control information between execution stages, updating protected architectural state, and performing security validation when instructions finally commit.

Because Sargantana is an out-of-order processor, Shadow Stack operations cannot modify architectural state immediately after instruction decode. Instead, all updates must remain speculative until the corresponding instruction successfully retires. This requirement ensures that incorrectly speculated instructions never corrupt the protected Shadow Stack.

Preparing the processor for these capabilities establishes the architectural interface that will later connect the Shadow Stack mechanism to each pipeline stage.

5.5 Hardware Resources

Implementing the Zicfiss extension requires several new hardware resources that operate together to provide secure return-address protection. The primary resource is the protected Shadow Stack memory used to store authenticated return addresses independently from the software stack. A dedicated Shadow Stack Pointer is required to manage this memory, while supporting control logic coordinates push and pop operations generated during function calls and returns.

Additional comparison logic is necessary to validate return addresses before control flow is transferred back to the calling function. Protection logic must also ensure that only authorized processor operations can modify the Shadow Stack, preventing accidental or malicious corruption of protected state.

5.6 Expected Outcome

At the completion of Phase 1, the overall architecture required for Zicfiss support will be fully defined. The processor will have a clear specification of the additional architectural state, protected memory structures, control mechanisms, and hardware resources necessary to support the Shadow Stack extension.

Although no pipeline stages are modified during this phase, every component required for implementation will have been identified and architecturally positioned. This preparation provides a well-defined foundation for the next phase, where the proposed Shadow Stack architecture will be integrated throughout the Sargantana processor pipeline.

Chapter 6

Phase 2 – Zicfiss Pipeline Integration

6.1 Phase Overview

6.1.1 Objectives

The objective of Phase 2 is to integrate the Zicfiss extension into the Sargantana processor pipeline without affecting the normal execution flow of existing RISC-V instructions. During this phase, the previously defined shadow stack architecture is connected to the processor pipeline so that every function call automatically stores the architectural return address onto the protected shadow stack, while every function return validates the return address before control is transferred.

The integration must preserve the processor's out-of-order execution model while ensuring that the architectural state is only updated when instructions commit. Therefore, the pipeline modifications focus on propagating shadow stack control information through the pipeline rather than changing the fundamental pipeline organization.

6.1.2 Pipeline Integration Strategy

Rather than introducing an independent execution path, the Zicfiss implementation extends the existing instruction flow. Each pipeline stage is responsible for handling a specific portion of the shadow stack operation.

The integration strategy consists of:

- Identifying CALL and RETURN instructions during instruction decoding.
- Propagating shadow-stack control signals through the rename and scheduling stages.
- Executing shadow stack push and pop operations together with the corresponding instruction.
- Delaying architectural state updates until retirement.
- Recovering speculative shadow-stack state whenever pipeline recovery occurs.

This approach minimizes hardware changes while maintaining compatibility with the current pipeline structure.

6.2 Instruction Fetch Integration (Phase 2A)

6.2.1 Existing Fetch Architecture

The Sargantana frontend consists of two independent fetch stages.

Fetch Stage 1 (IF1) is responsible for program counter generation, instruction cache access, branch prediction, branch target buffer lookup, and return address stack prediction.

Fetch Stage 2 (IF2) receives the fetched instruction, performs preliminary instruction buffering, and forwards instructions toward the decoder.

Since Zicfiss protects function returns after instruction decoding, neither IF1 nor IF2 requires fundamental architectural modifications. Their existing responsibility of instruction delivery remains unchanged.

6.2.2 RTL Files to be Modified

For the fetch subsystem, only minimal integration support is required.

Primary RTL Files

RTL File	Modification
`if_stage_1/rtl/if_stage_1.sv`	Minor interface updates (if additional pipeline metadata is propagated).
`if_stage_2/rtl/if_stage_2.sv`	Pipeline signal forwarding if required.
`datapath.sv`	Additional inter-stage signal routing for Zicfiss metadata.

Files Remaining Unchanged

The following modules continue operating exactly as before because Zicfiss does not alter instruction fetching or branch prediction:

- `branch_predictor.sv`
- `bimodal_predictor.sv`
- `return_address_stack.sv`
- Instruction cache logic
- Branch Target Buffer
- Branch History Table

These modules already predict return targets and are independent of shadow stack verification, which occurs later in the pipeline.

6.2.3 New Hardware Components

No new functional hardware units are introduced within the fetch subsystem.

The existing fetch architecture is reused entirely.

If implementation requires additional metadata propagation, only small pipeline registers or control fields are added without affecting the instruction fetch datapath.

6.2.4 Components Remaining Unchanged

The following hardware remains identical to the original Sargantana implementation:

- Program Counter generation logic
- Instruction Cache
- Fetch pipeline timing
- Branch Prediction Unit
- Branch Target Buffer (BTB)
- Branch History Table (BHT)
- Return Address Stack predictor
- Instruction fetch bandwidth
- Instruction cache interface

Consequently, the fetch stage continues operating at the original throughput without introducing additional latency.

6.2.5 Testbench Updates

The existing fetch-stage verification environment can largely be reused.

Existing Testbenches

Testbench	Purpose
`tb_if_stage` (IF1)	Verify normal instruction fetch behaviour after interface updates.
`tb_if_stage` (IF2)	Verify correct forwarding of instruction metadata.
`tb_branch_predictor`	Regression testing to ensure prediction accuracy is unaffected.

Required Verification Changes

The following directed tests cases should be added in the existing tb:

- Normal instruction fetch with Zicfiss enabled.
- CALL instruction entering the pipeline.
- RETURN instruction entering the pipeline.
- Pipeline flush after CALL.
- Branch misprediction recovery.
- Speculative instruction cancellation.
- Continuous mixed instruction stream.

These tests verify that the frontend remains functionally identical while correctly forwarding all information required by the downstream Zicfiss logic.

6.3 Instruction Decode Integration (Phase 2B)

6.3.1 Existing Decode Architecture

The Decode (ID) stage is responsible for translating the fetched instruction into internal control signals that drive the remainder of the processor pipeline. In Sargantana, this stage receives instructions from Fetch Stage 2 and performs opcode decoding, immediate extraction, instruction classification, operand identification, and generation of execution control information before dispatching the instruction into the Instruction Queue.

For the existing implementation, the Decode stage mainly consists of:

- Instruction decoder
- Immediate generation logic
- Vector instruction decoder (`vset_module`)
- Vector configuration queue (`vset_queue`)

After decoding, all generated control information is forwarded toward the Instruction Queue and Rename stage.

Since **Zicfiss introduces three new instructions (SSPUSH, SSPOPCHK and SSRD)** together with shadow stack semantics, the Decode stage becomes the **first architectural point where Shadow Stack instructions are recognized**. Unlike ordinary load/store instructions, these instructions must be tagged as security operations so that later pipeline stages can route them toward the Shadow Stack hardware rather than the normal memory hierarchy.

6.3.2 RTL Files to be Modified

Unlike Fetch, Decode requires modifications across several interconnected RTL modules because instruction classification propagates throughout the pipeline.

Primary RTL Files

RTL File	Modification	Reason
`rtl/datapath/id_stage/rtl/decoder.sv`	**Major**	Decode SSPUSH, SSPOPCHK and SSRD opcodes and generate Shadow Stack control signals.
`rtl/control_unit/rtl/control_unit.sv`	**Moderate**	Add new control signals corresponding to Shadow Stack instructions and propagate them through the datapath.
`rtl/datapath/rtl/datapath.sv`	**Moderate**	Extend decode-to-rename pipeline registers to carry new Zicfiss metadata.

Supporting RTL Files (Indirect Changes)

Although their functionality does not fundamentally change, these modules require interface updates because new decode signals are introduced.

RTL File	Modification	Purpose
`immediate.sv`	Minor	Ensure immediate extraction supports any immediate formats required by Shadow Stack instructions (if applicable).
`instruction_queue.sv`	Interface update	Store newly generated Shadow Stack control bits alongside each instruction.
`rename_table.sv`	Interface update	Receive Shadow Stack instruction classification for later Rename handling.
`free_list.sv`	No functional change	Interface remains compatible after metadata propagation.

6.3.3 Required Architectural Changes

To support Zicfiss, several architectural enhancements are required during instruction decoding.

A. Recognition of New ISA Instructions

The decoder must identify:

- SSPUSH (shadow stack PUSH)
- SSPOPCHK (Shadow Stack POP and CHECK)
- SSRD (Shadow stack READ)

using their opcode and function field combinations defined by the RISC-V Zicfiss specification.

These instructions should no longer be treated as illegal or unknown operations.

B. Shadow Stack Instruction Classification

After decoding, every instruction should be classified as either:

- Normal instruction
- Shadow Stack instruction

A dedicated control bit such as:

is_shadow_stack

should accompany the instruction through the remaining pipeline.

C. Operation Identification

The decoder should additionally generate the operation type.

Example:

shadow_stack_op

00 -> None
01 -> SSPUSH
10 -> SSPOPCHK
11 -> SSRD

This avoids repeated opcode decoding in downstream stages.

D. Privilege Validation Metadata

The decoder should also generate metadata indicating that the instruction requires privilege validation before execution.

Actual privilege enforcement is **not performed here**; instead, the information is propagated for later security checking.

E. Exception Classification

Illegal instruction detection logic should be extended so that:

- valid Zicfiss instructions are accepted,
- malformed encodings continue to raise Illegal Instruction exceptions.

6.3.4 New Hardware Components

Although Decode already contains the required infrastructure, several small logic blocks should be added.

A. Shadow Stack Instruction Decoder

New file

shadow_stack_decoder.sv

Purpose:

- Decode Zicfiss instruction encodings.
- Generate internal operation identifiers.
- Produce security control signals.

Outputs include:

- `is_shadow_stack`
- `shadow_stack_op`
- `requires_ssp`
- `requires_privilege_check`

Separating this functionality keeps the main decoder modular.

B. Shadow Stack Control Signal Bundle

A new packed structure (or equivalent control bundle) should be introduced to carry Shadow Stack metadata through pipeline registers.

Typical fields include:

- instruction type
- Shadow Stack operation
- privilege requirement
- security flags

C. Decode Security Validation Logic

A lightweight combinational block should verify:

- reserved encodings,
- unsupported combinations,
- invalid instruction formats.

This reduces unnecessary work in later stages.

6.3.5 Components Remaining Unchanged

The following Decode components remain architecturally unchanged because Zicfiss does not affect their functionality.

Module	Reason
`vset_module.sv`	Vector configuration instructions are unrelated to Shadow Stack.
`vset_queue.sv`	Handles vector state only.
Existing immediate arithmetic logic	Existing arithmetic immediate generation remains unchanged except optional interface extensions.
Normal ALU decode paths	Integer arithmetic decoding remains identical.
Floating-point decode	No interaction with Shadow Stack.
SIMD decode	Independent extension.

6.3.6 Testbench Updates

Decode verification must be expanded to ensure correct recognition and classification of all Zicfiss instructions.

Existing Testbench to Modify

rtl/datapath/id_stage/tb/tb_decoder/tb_decoder.sv

New directed tests should verify:

- decoding of SSPUSH,
- decoding of SSPOPCHK,
- decoding of SSRD,
- generation of Shadow Stack control signals,
- correct operation identifier generation,

- illegal instruction detection,
- reserved encoding rejection,
- propagation of security metadata toward the Instruction Queue.

Coverage should also ensure that normal RISC-V instructions continue to decode exactly as before.

New Testbench

tb_shadow_stack_decoder.sv

Purpose:

- Unit testing of the new `shadow_stack_decoder.sv`.
- Verification of every legal Zicfiss encoding.
- Verification of illegal opcode combinations.
- Validation of generated security metadata independent of the full Decode pipeline.

6.4 Rename Stage Integration (Phase 2C)

6.4.1 Existing Rename Architecture

The Rename stage removes false register dependencies by translating architectural registers into physical registers. In Sargantana, this stage contains three primary architectural blocks:

- Instruction Queue
- Rename Table
- Free List

The Rename Table maintains architectural-to-physical register mappings, while the Free List manages allocation of unused physical registers. Once register renaming is complete, renamed instructions are forwarded toward the Register Read stage.

For ordinary instructions, Rename modifies destination register mappings before execution.

For Zicfiss, however, Shadow Stack instructions **do not allocate new architectural register destinations** (except SSRD, which writes the shadow stack pointer into a general-purpose register). Consequently, Rename must recognize these instructions and avoid unnecessary physical register allocation.

6.4.2 RTL Files to be Modified

Primary RTL Files

RTL File	Modification	Reason
`rtl/datapath/ir_stage/rtl/instruction_queue.sv`	Major	Store and forward Shadow Stack metadata generated during Decode.
`rtl/datapath/ir_stage/rtl/rename_table.sv`	Major	Prevent unnecessary register renaming for SSPUSH and SSPOPCHK while allowing SSRD destination renaming.
`rtl/datapath/rtl/datapath.sv`	Moderate	Extend Rename pipeline registers with Shadow Stack control signals.

Supporting RTL Files

RTL File	Modification	Reason
`free_list.sv`	Minor	Prevent physical register allocation for instructions without destination registers.
`control_unit.sv`	Interface update	Forward Rename control information for Shadow Stack instructions.

6.4.3 Required Architectural Changes

A. Preserve Security Metadata

Rename must forward:

- Shadow Stack operation
- privilege metadata
- security flags

without modification.

B. Destination Register Handling

Rename behaviour should depend on instruction type.

Instruction	Rename Required
SSPUSH	No

SSPOPCHK	No
SSRD	Yes

Only SSRD allocates a destination physical register.

C. Free List Control

The Free List should avoid allocating physical registers for instructions that do not produce architectural results.

This improves efficiency and prevents unnecessary physical register consumption.

D. Instruction Queue Extension

Each queued instruction should additionally store:

- Shadow Stack operation
- security metadata
- privilege requirement

These fields accompany the instruction until execution.

E. Recovery Compatibility

Rename recovery mechanisms (used during pipeline flushes and branch recovery) should preserve correct handling of pending Shadow Stack instructions to maintain architectural consistency after speculation recovery.

6.4.4 New Hardware Components

A. Shadow Stack Rename Controller

New file

shadow_stack_rename.sv

Responsibilities:

- Determine whether physical register allocation is required.
- Generate Rename control decisions.
- Distinguish between SSPUSH, SSPOPCHK and SSRD.

B. Shadow Stack Metadata Buffer

A small metadata register or packed structure should accompany renamed instructions, storing:

- Shadow Stack operation
- security flags
- privilege information
- destination register requirement

C. Rename Allocation Filter

A lightweight combinational block should gate Free List allocation requests whenever a Shadow Stack instruction does not require a destination register.

6.4.5 Components Remaining Unchanged

Module	Reason
Physical register file organization	No structural changes required.
Register renaming algorithm	Existing renaming mechanism remains unchanged.
Rename checkpoint logic	Existing recovery mechanism remains valid.
General dependency tracking	Shadow Stack instructions follow existing dependency rules.
Issue arbitration	Scheduling policy does not change.

6.4.6 Testbench Updates

Existing Testbenches to Modify

- `rtl/datapath/ir_stage/tb/tb_rename/tb_rename_table.sv`
- `rtl/datapath/ir_stage/tb/tb_free_list/tb_free_list.sv`

New test scenarios should include:

- SSPUSH performs no destination register allocation.
- SSPOPCHK performs no destination register allocation.
- SSRD allocates exactly one physical register.
- Instruction Queue stores and forwards Shadow Stack metadata correctly.
- Free List remains unchanged after non-register-writing Shadow Stack instructions.

- Rename recovery correctly handles in-flight Shadow Stack instructions during pipeline flushes.

New Testbench (Recommended)

tb_shadow_stack_rename.sv

This dedicated testbench should verify the new Rename control logic independently, including allocation decisions, metadata propagation, and interaction with the Free List before integration with the complete pipeline.

6.5 Register Read Stage Integration (Phase 2D)

6.5.1 Existing Register Read Architecture

The Register Read (RR) stage is responsible for reading operands from the physical register file after register renaming has been completed. At this point, architectural registers have already been translated into physical registers by the Rename stage, allowing the Register Read stage to fetch the required source operands before dispatching the instruction to the execution units.

In the current Sargantana implementation, the Register Read stage mainly consists of the Physical Register File together with operand bypassing and hazard detection mechanisms. The stage supplies operands to all execution units including the ALU, Branch Unit, Memory Unit, Multiplier, Divider, Floating Point Unit, and SIMD execution pipelines.

For Zicfiss, this stage does not perform any security verification. Instead, it prepares the required operands for Shadow Stack instructions. In particular, **SSPUSH** requires the return address operand, **SSPOPCHK** requires the architectural return address for comparison, while **SSRD** does not require a memory operand but later produces the current Shadow Stack Pointer value.

6.5.2 RTL Files to be Modified

Primary RTL Files

RTL File	Modification	Reason
`rtl/datapath/rr_stage/rtl/regfile.sv`	Moderate	Provide operands required by Shadow Stack instructions and propagate Shadow Stack metadata toward the Execution stage.
`rtl/datapath/rtl/datapath.sv`	Moderate	Extend RR-to-EXE pipeline registers to carry Shadow Stack control information.

Supporting RTL Files

RTL File	Modification	Reason
`rtl/control_unit/rtl/control_unit.sv`	Interface update	Forward Shadow Stack control signals into the Register Read stage.
`rtl/datapath/ir_stage/rtl/instruction_queue.sv`	Interface update	Deliver decoded Shadow Stack metadata together with renamed operands.
`rtl/datapath/ir_stage/rtl/rename_table.sv`	No functional modification	Existing physical register mapping remains valid.

6.5.3 New Hardware Components

Unlike Decode and Rename, the Register Read stage requires very little new hardware.

A. Shadow Stack Operand Selector

New file

shadow_stack_operand_select.sv

Responsibilities:

- Select the correct source register for SSPUSH.
- Select the comparison operand for SSPOPCHK.
- Bypass unnecessary operand reads for SSRD.

Separating this logic avoids adding excessive complexity inside `regfile.sv`.

B. Shadow Stack Metadata Register

A small pipeline register should be introduced to accompany the instruction into the Execution stage.

Stored information includes:

- Shadow Stack operation
- privilege information
- destination register requirement
- security flags

6.5.4 Components Remaining Unchanged

Module	Reason
Physical Register File implementation	Existing register storage remains unchanged.
Register bypass network	Shadow Stack instructions use the existing forwarding mechanism.
Hazard detection logic	Existing RAW/WAW handling is sufficient.
Register allocation logic	Already handled during Rename.
Operand forwarding logic	No additional forwarding paths are required.

6.5.5 Testbench Updates

Existing Testbench to Modify

rtl/datapath/rr_stage/tb/tb_regfile/tb_regfile.sv

New directed tests should verify:

- correct operand read for SSPUSH
- correct operand read for SSPOPCHK
- no unnecessary register read for SSRD
- correct propagation of Shadow Stack metadata
- compatibility with existing bypass logic
- interaction with renamed physical registers

New Testbench

tb_shadow_stack_operand_select.sv

Purpose:

- Verify operand selection independently.
- Validate source register selection.

- Verify metadata propagation.
- Ensure SSRD correctly bypasses unnecessary operand reads.

6.6 Execution Stage Integration (Phase 2E)

6.6.1 Existing Execution Architecture

The Execution stage is the computational core of the Sargantana processor. After operands are obtained from the Register Read stage, instructions are dispatched to the appropriate execution unit, including the Integer ALU, Branch Unit, Memory Unit, Multiplier, Divider, Floating Point Unit, SIMD Unit, and Load/Store subsystem.

The stage also contains scoreboards, pending operation queues, memory request handling, and address generation logic, allowing multiple execution pipelines to operate simultaneously.

For the Zicfiss extension, the Execution stage becomes the **primary enforcement point** of the Shadow Stack mechanism. While previous pipeline stages merely recognize and propagate Shadow Stack instructions, the Execution stage performs the actual security operations including updating the Shadow Stack Pointer, reading and writing Shadow Stack memory, validating return addresses, detecting violations, and generating architectural exceptions.

6.6.2 RTL Files to be Modified

Core Execution Files

RTL File	Modification	Reason
`rtl/datapath/exe_stage/rtl/exe_stage.sv`	**Major**	Integrate Shadow Stack execution pipeline and dispatch new Shadow Stack instructions.
`rtl/datapath/exe_stage/rtl/mem_unit.sv`	**Major**	Add support for accesses directed to Shadow Stack memory rather than normal data memory.
`rtl/datapath/exe_stage/rtl/load_store_queue.sv`	Moderate	Prevent ordinary memory scheduling rules from interfering with protected Shadow Stack operations.

`rtl/datapath/exe_stage/rtl/store_buffer.sv`	Moderate	Handle pending Shadow Stack write operations correctly.
`rtl/datapath/exe_stage/rtl/vagu.sv`	Minor	Generate addresses for Shadow Stack memory accesses where required.
`rtl/datapath/rtl/datapath.sv`	Moderate	Carry Shadow Stack execution results toward the Writeback stage.
`rtl/control_unit/rtl/control_unit.sv`	Moderate	Add execution control signals for Shadow Stack operations.
`rtl/datapath/interface_csr/rtl/csr_interface.sv`	Moderate	Connect execution logic with Shadow Stack CSRs (e.g., SSP).

Supporting Modules

The following execution modules are not directly modified but must interface correctly with the new execution path.

- `score_board_scalar.sv`
- `pending_mem_req_queue.sv`
- `pending_fp_ops_queue.sv`

These modules primarily require interface updates so Shadow Stack operations are tracked correctly within the execution pipeline.

6.6.3 Required Architectural Changes

A. Dedicated Shadow Stack Execution Path

A dedicated execution path should be introduced inside `exe_stage.sv` to dispatch:

- SSPUSH
- SSPOPCHK
- SSRD

to dedicated Shadow Stack hardware rather than the ordinary ALU or memory pipelines.

B. Shadow Stack Pointer Management

Execution hardware should maintain the Shadow Stack Pointer by:

- incrementing after successful pushes,
- decrementing during return verification,
- reading SSP for SSRD,
- validating pointer alignment and bounds.

C. Protected Shadow Stack Memory Access

Memory accesses generated by Shadow Stack instructions must be redirected toward protected Shadow Stack memory rather than the normal Data Cache hierarchy.

This prevents ordinary software from modifying protected return addresses.

D. Return Address Verification

During SSPOPCHK:

1. Read protected Shadow Stack entry.
2. Compare with architectural return address.
3. Detect mismatch.
4. Generate security exception on failure.

E. Exception Generation

Execution hardware must generate exceptions for:

- Shadow Stack mismatch
- invalid Shadow Stack Pointer
- privilege violations
- illegal Shadow Stack accesses
- stack underflow
- stack overflow

F. CSR Communication

Execution logic should communicate with the CSR subsystem to:

- read SSP
- update SSP
- report security exceptions
- synchronize privilege state

6.6.4 New Hardware Components

This stage introduces most of the new RTL required for Zicfiss.

A. Shadow Stack Execution Unit

New file

shadow_stack_unit.sv

Responsibilities:

- Execute SSPUSH.
- Execute SSPOPCHK.
- Execute SSRD.
- Generate execution results.
- Interface with Shadow Stack memory.

B. Shadow Stack Pointer Register

New file

ssp_register.sv

Responsibilities:

- Store current SSP.
- Increment/decrement pointer.
- Perform alignment checking.
- Detect overflow and underflow.

C. Shadow Stack Memory Controller

New file

shadow_stack_memory_controller.sv

Responsibilities:

- Read Shadow Stack entries.
- Write Shadow Stack entries.
- Enforce protected memory accesses.
- Interface with memory subsystem.

D. Shadow Stack Comparator

New file

shadow_stack_compare.sv

Responsibilities:

- Compare return addresses.
- Detect mismatches.
- Generate comparison result.

E. Shadow Stack Exception Generator

New file

shadow_stack_exception.sv

Responsibilities:

- Generate architectural exceptions.
- Report security violations.
- Interface with CSR exception handling.

F. Shadow Stack CSR Controller (Not Sure , like first have to check current CSR)

New file

shadow_stack_csr_controller.sv

Responsibilities:

- Coordinate SSP CSR accesses.
- Synchronize privilege state.
- Update Shadow Stack CSRs.

6.6.5 Components Remaining Unchanged

Module	Reason
Integer ALU (`alu.sv` and submodules)	Arithmetic functionality remains unchanged.
Branch Unit (`branch_unit.sv`)	Branch prediction and resolution are unaffected.
Multiplier (`mul_unit.sv`)	Independent execution pipeline.
Divider (`div_unit.sv`)	Independent execution pipeline.
Floating Point Unit	No interaction with Shadow Stack.
SIMD execution units	Vector execution is unaffected.
Floating-point pending queues	Unrelated to Shadow Stack operations.

SIMD scoreboards	Existing vector dependency tracking remains unchanged.

6.6.6 Testbench Updates

Existing Testbenches to Modify

The following execution testbenches should be extended to verify Shadow Stack integration:

- `tb_mem_unit.sv`
- `tb_ls_queue.sv`
- `tb_top/tb_module.sv`

These testbenches should include scenarios for:

- SSPUSH execution
- SSPOPCHK execution
- SSRD execution
- correct Shadow Stack memory accesses
- SSP updates after push and pop
- return address comparison
- stack overflow detection
- stack underflow detection
- privilege violations
- exception generation
- execution pipeline stalls
- interaction with memory subsystem

New Testbenches

tb_shadow_stack_unit.sv
tb_shadow_stack_memory_controller.sv
tb_ssp_register.sv
tb_shadow_stack_compare.sv
tb_shadow_stack_exception.sv
tb_shadow_stack_csr_controller.sv

These unit-level testbenches should verify each newly introduced hardware component independently before validating the complete execution pipeline through the existing top-level execution testbench.

6.7 Writeback and Graduation (Commit) Integration (Phase 2F)

6.7.1 Existing Commit Architecture

The Writeback and Graduation (Commit) stages represent the final phase of instruction execution within the Sargantana pipeline. During Writeback, execution results produced by the functional units are written back to the physical register file. The Graduation stage then retires instructions in program order, updates the architectural state, releases physical registers, and ensures precise exception handling.

The Graduation List (`graduation_list.sv`) maintains the order of in-flight instructions and guarantees that instructions are committed only after successful execution. This stage is also responsible for handling pipeline flushes, exceptions, and recovery following branch mispredictions or execution faults.

For the Zicfiss extension, Shadow Stack instructions must also retire in program order. Any detected Shadow Stack violation must prevent the instruction from committing, trigger an architectural exception, and preserve precise processor state.

6.7.2 RTL Files to be Modified

Primary RTL Files

RTL File	Modification	Reason
`rtl/datapath/wb_stage/rtl/graduation_list.sv`	**Major**	Support retirement of Shadow Stack instructions and handle Shadow Stack exceptions.
`rtl/datapath/rtl/datapath.sv`	Moderate	Propagate Shadow Stack execution results into the Writeback and Graduation stages.
`rtl/control_unit/rtl/control_unit.sv`	Moderate	Handle commit control and pipeline flushes generated by Shadow Stack exceptions.

Supporting RTL Files

RTL File	Modification	Reason
`rtl/datapath/exe_stage/rtl/exe_stage.sv`	Interface update	Forward Shadow Stack completion status.

`rtl/datapath/interface_csr/rtl/csr_interface.sv`	Interface update	Commit SSP updates and exception information.

6.7.3 Required Architectural Changes

A. Shadow Stack Instruction Retirement

The Graduation stage must recognize completed Shadow Stack instructions and retire them in program order while preserving precise architectural state.

B. Exception-Aware Commit

If SSPOPCHK detects a mismatch or another Shadow Stack exception is generated, the corresponding instruction must not commit. Instead, the Graduation stage should:

- stop retirement,
- flush younger instructions,
- preserve architectural state,
- invoke the processor's exception handling mechanism.

C. SSP Commit Synchronization

Updates to the Shadow Stack Pointer (SSP) must become architecturally visible only after successful retirement of the corresponding Shadow Stack instruction.

D. Pipeline Recovery

The existing recovery mechanism should be extended to recover correctly from:

- Shadow Stack mismatch
- Stack overflow
- Stack underflow
- Privilege violations
- Illegal Shadow Stack operations

6.7.4 New Hardware Components

A. Shadow Stack Commit Controller

New file

shadow_stack_commit.sv

Responsibilities:

- Retire Shadow Stack instructions.
- Coordinate SSP updates.
- Validate successful execution before commit.
- Interface with the Graduation List.

B. Shadow Stack Recovery Controller

New file

shadow_stack_recovery.sv

Responsibilities:

- Flush younger instructions.
- Restore architectural state.
- Recover pipeline after Shadow Stack exceptions.
- Coordinate exception handling.

6.7.5 Components Remaining Unchanged

Module	Reason
Physical Register File	Existing writeback mechanism remains unchanged.
Register release mechanism	Physical register reclamation continues normally.
Ordinary instruction retirement	Existing commit logic remains unchanged for non-Shadow Stack instructions.
ALU/FPU/SIMD writeback paths	No modifications required.
Floating-point commit logic	Independent of Shadow Stack execution.

6.7.6 Testbench Updates

Existing Testbench to Modify

rtl/datapath/wb_stage/tb/tb_graduation_list/tb_graduation_list.sv

New test cases should verify:

- Successful retirement of SSPUSH
- Successful retirement of SSPOPCHK
- Successful retirement of SSRD
- Correct SSP update after commit
- Prevention of commit following Shadow Stack mismatch

- Pipeline flush after security exception
- Correct recovery after exception
- Interaction with existing graduation mechanism

New Testbenches

tb_shadow_stack_commit.sv
tb_shadow_stack_recovery.sv

These unit-level testbenches should validate the newly introduced commit controller and recovery controller before integration with the complete Graduation stage.

6.8 Phase Summary

6.8.1 Overall Pipeline Changes

Phase 2 integrates the Zicfiss Shadow Stack extension throughout the complete Sargantana pipeline, beginning with instruction fetch and ending at architectural commit. Rather than modifying only a single execution unit, the implementation introduces coordinated support across every major pipeline stage to ensure secure handling of Shadow Stack instructions while maintaining pipeline correctness and precise exception handling.

The overall integration can be summarized as follows:

Pipeline Stage	Primary Modified RTL Files	Major Functionality Added
Instruction Fetch (IF1 & IF2)	`if_stage_1.sv`, `if_stage_2.sv`, `branch_predictor.sv`, `return_address_stack.sv`, `datapath.sv`	Fetch and recognize Shadow Stack instructions
Instruction Decode (ID)	`decoder.sv`, `immediate.sv`, `control_unit.sv`, `datapath.sv`	Decode SSPUSH, SSPOPCHK, SSRD
Rename (IR)	`instruction_queue.sv`, `rename_table.sv`, `free_list.sv`, `datapath.sv`	Rename and propagate Shadow Stack metadata
Register Read (RR)	`regfile.sv`, `datapath.sv`	Read operands and forward Shadow Stack information

Execution (EXE)	`exe_stage.sv`, `mem_unit.sv`, `load_store_queue.sv`, `store_buffer.sv`, `vagu.sv`, `csr_interface.sv`	Execute Shadow Stack operations, compare return addresses, update SSP, generate exceptions
Writeback & Graduation (WB)	`graduation_list.sv`, `datapath.sv`, `control_unit.sv`	Commit Shadow Stack instructions and recover from security exceptions

6.8.2 Phase Completion Summary

At the completion of Phase 2, support for the Zicfiss Shadow Stack extension has been integrated across all major stages of the Sargantana processor pipeline. Instruction fetch, decode, rename, register read, execution, and retirement stages have been extended to recognize, execute, and securely commit Shadow Stack instructions while preserving compatibility with the existing out-of-order microarchitecture.

Overall, Phase 2 establishes a complete pipeline-level implementatio framework for the Zicfiss extension and provides the architectural foundation required for functional verification, performance evaluation, and subsequent hardware validation in later project phases.

Topics	Covered in Chapter 6 Section	Description
Return Address Verification	**Section 6.6 – Execution Stage Integration** (6.6.3 Required Architectural Changes)	Return address comparison between the architectural stack and Shadow Stack is performed during execution using the proposed comparison logic.
Fault Detection Logic	**Section 6.6 – Execution Stage Integration** (6.6.3 & 6.6.4)	Shadow Stack mismatch detection, stack overflow/underflow detection, and security fault identification are implemented within the execution stage.
Exception Generation	**Section 6.6 – Execution Stage Integration** (6.6.3)	Security exceptions generated by Shadow Stack violations are produced during execution and propagated through the pipeline.
Pipeline Recovery	**Section 6.7 – Writeback and Graduation Integration** (6.7.3 & 6.7.4)	Pipeline flush, architectural state recovery, and precise exception handling are performed during instruction retirement and commit.

Chapter 7

Phase 3 – Zicfiss Verification and Validation

7.1 Verification Objectives

- Verify correct implementation of the Zicfiss extension.
- Ensure proper Shadow Stack functionality.
- Validate pipeline integration without affecting existing processor behavior.

7.2 RTL Simulation

- Verify individual RTL modules using SystemVerilog testbenches.
- Simulate using Verilator or QuestaSim.

7.3 Directed Tests

- Execute hand-written assembly programs.
- Test valid and invalid function returns.
- Verify SSPUSH and SSPOPCHK operations.

7.4 ISA Compliance Tests

- Run official RISC-V CFI compliance tests.
- Verify architectural correctness of the implementation.

7.5 Linux / Emulator Validation

- Execute programs on the Sargantana emulator.
- Confirm correct processor behavior under realistic workloads.

7.6 Performance Evaluation

- Measure hardware overhead.
- Evaluate execution latency and pipeline impact.

7.7 Expected Outcome

- Functional and ISA-compliant Zicfiss implementation.
- Successful validation across RTL, compliance, and system-level testing.

7.8 Summary

Verification Type	What is Tested	How it is Performed	Goal
RTL Simulation	Individual RTL modules	SystemVerilog testbenches (Verilator/QuestaSim)	Verify module functionality
Directed Tests	Zicfiss instruction sequences	Hand-written assembly programs	Verify Shadow Stack features
ISA Compliance Tests	ISA correctness	Official RISC-V CFI test suite	Ensure specification compliance
Linux / Emulator Tests	Complete processor	Run software on the Sargantana emulator	Validate real-world operation
Performance Evaluation	Pipeline overhead	Compare execution before and after integration	Measure security overhead

Chapter 8

Phase 4 – Zicfilp Foundation and Architectural Preparation

8.1 Objectives

The objective of this phase is to study the Zicfilp (Control-flow Landing Pad) ISA extension and understand how it fits within the existing Sargantana processor architecture. This phase identifies the processor components that participate in indirect control transfers, defines the architectural state required by Zicfilp, analyzes the impact on the instruction pipeline, and prepares the RTL design for subsequent hardware implementation.

8.2 Existing Control Flow Analysis

Before integrating Zicfilp, the current control-flow mechanism of Sargantana must be analyzed. The processor already supports indirect branches, indirect jumps (`JALR`), function calls, and returns through its branch prediction and fetch pipeline. During this phase, the complete execution path of an indirect branch is studied to determine where landing pad verification can be introduced with minimum impact on the existing pipeline.

The interaction between the Fetch stages, Decode stage, Execute stage, Branch Unit, and Commit logic is examined to identify where the processor currently resolves indirect control transfers and where additional security checks should be inserted.

8.3 Landing Pad Architecture

The Zicfilp extension protects indirect control transfers by ensuring that every valid indirect branch target begins with an `LPAD` instruction. During this phase, the complete landing pad mechanism is studied, including the Expected Landing Pad (ELP) state, the execution flow of the `LPAD` instruction, and the exception behavior when a landing pad is missing or invalid.

The interaction between software-generated landing pads and hardware verification logic is analyzed to determine how the processor should validate indirect branch targets before allowing normal execution to continue.

8.4 Architectural State Requirements

The following architectural resources(Expected) are to be identified:

- Expected Landing Pad (ELP) state
- Landing Pad Enable control signal
- LPAD instruction detection logic
- Illegal landing pad exception conditions
- Pipeline flush and recovery signals
- Additional decode status bits for indirect branches

These resources will later be integrated into the appropriate pipeline stages during the implementation phase.

8.5 Pipeline Preparation

The processor pipeline is analyzed stage-by-stage to determine the modifications required for Zicfilp support.

The primary stages involved are:

- **Instruction Fetch (IF1 & IF2):** Fetch target instructions after indirect branches.
- **Instruction Decode (ID):** Detect `LPAD` instructions and decode Zicfilp-specific behavior.
- **Instruction Rename (IR):** Propagate Zicfilp control information through the out-of-order pipeline.
- **Register Read (RR):** Preserve architectural state required for execution.
- **Execute (EX):** Verify landing pad correctness and generate exceptions on failures.
- **Writeback / Graduation (WB):** Commit verified instructions while ensuring precise exception handling.

This analysis establishes the roadmap for the detailed RTL modifications presented in the following implementation phase.

8.6 Hardware Resources

Based on the architectural analysis, the following hardware components are identified as necessary for Zicfilp implementation.

Hardware Resource	Purpose
Expected Landing Pad (ELP) Register	Tracks whether the next fetched instruction must be an LPAD
LPAD Detection Logic	Detects valid landing pad instructions during decode
Landing Pad Verification Logic	Confirms indirect branch targets begin with LPAD
Exception Generation Logic	Raises Control Flow Integrity exceptions on invalid targets

Pipeline Flush Controller	Flushes incorrect instructions after a violation
Additional Pipeline Control Signals	Propagate Zicfilp state between stages

These components form the foundation for the RTL implementation described in the next phase.

8.7 Expected Outcome

At the completion of this phase, the architectural behavior of the Zicfilp extension will be fully understood and mapped onto the Sargantana processor. The existing control-flow mechanism, affected pipeline stages, required architectural state, and necessary hardware resources will be clearly identified. This preparation provides the foundation required for the RTL integration of Zicfilp in the following implementation phase.

Chapter 9

Phase 5 – Zicfilp Pipeline Integration

9.1 Phase Overview

9.1.1 Objectives

The primary objectives of this phase are:

- Integrate Zicfilp support into the processor pipeline without affecting normal instruction execution.
- Detect indirect control-flow instructions during instruction decoding.
- Verify that the destination of every indirect jump begins with a valid **LPAD** instruction.
- Propagate CFI-related information across the pipeline until verification completes.
- Generate architectural exceptions for invalid landing pads.
- Preserve compatibility with the existing speculation, branch prediction, and out-of-order execution mechanisms.

9.1.2 Pipeline Integration Strategy

Rather than introducing an independent CFI pipeline, Zicfilp functionality is incorporated into the existing processor stages.

The fetch stage supplies instruction bytes from the predicted target as before. During decode, the processor identifies indirect control-flow instructions together with LPAD instructions. Rename and register-read stages propagate CFI metadata through existing pipeline structures. The execution stage performs landing-pad validation before allowing architectural state updates, while the commit stage retires only successfully verified indirect branches.

9.2 Instruction Fetch Integration (Phase 5A)

9.2.1 Existing Fetch Architecture

For ordinary instruction execution, the fetch unit simply retrieves instructions from the predicted program counter without validating whether the fetched target corresponds to a legal landing pad.

For Zicfilp, this behavior must be extended so that **indirect branch targets** eventually reach the decoder together with sufficient metadata for landing-pad verification.

9.2.2 RTL Files to be Modified

The following RTL files are expected to require modifications during Zicfilp fetch-stage integration.

RTL File	Reason for Modification
`rtl/datapath/rtl/if_stage_1/rtl/if_stage_1.sv`	Primary fetch pipeline stage. Propagate CFI information alongside fetched instructions and predicted targets.
`rtl/datapath/rtl/if_stage_2/rtl/if_stage_2.sv`	Preserve LPAD-related metadata while forwarding instructions to the decode stage.
`rtl/datapath/rtl/datapath.sv`	Add new pipeline signals carrying CFI metadata between IF, ID and later stages.
`rtl/control_unit/rtl/control_unit.sv`	Extend global control logic to support fetch flushing and recovery following LPAD verification failures.

Existing Fetch Components Influenced

Although these modules may not require major RTL changes, they interact directly with the fetch stage and therefore must be reviewed during integration.

Module	Purpose
`branch_predictor.sv`	Continues predicting indirect branch targets that will later undergo LPAD verification.
`bimodal_predictor.sv`	Existing prediction mechanism remains operational without algorithmic changes.
`return_address_stack.sv`	Used for return prediction; operates normally alongside Zicfilp support.

9.2.3 Required Architectural Changes

The fetch stage requires several architectural enhancements to support landing-pad enforcement.

- Introduce a pipeline flag indicating that the fetched instruction originates from an indirect control-flow target.
- Preserve predicted target PC information until LPAD verification completes.
- Carry CFI metadata together with the fetched instruction packet.

- Ensure fetch packets distinguish ordinary sequential fetches from indirect control-flow fetches.
- Support pipeline flush requests generated after LPAD verification failures.
- Maintain compatibility with speculative execution and branch prediction.

No modification to instruction cache organization or fetch bandwidth is required.

9.2.4 New Hardware Components

Only lightweight hardware additions are expected during fetch-stage integration.

Component	Purpose
CFI Metadata Pipeline Register	Stores LPAD-related information alongside fetched instructions.
Indirect Branch Indicator	Marks instruction packets generated from indirect control-flow targets.
Target PC Metadata Register	Preserves predicted destination address until validation completes.
LPAD Fetch Status Signal	Indicates that the fetched instruction must undergo landing-pad verification in later stages.

These additions primarily consist of extra registers and pipeline signals rather than large functional units.

9.2.5 Components Remaining Unchanged

The following fetch components continue operating without functional modification.

- Instruction cache interface
- Fetch bandwidth
- Sequential PC increment logic
- Existing branch prediction algorithms
- Bimodal predictor
- Return Address Stack prediction algorithm
- Instruction alignment logic
- Fetch queue organization

These blocks continue providing instructions exactly as before, while Zicfilp verification occurs later in the pipeline.

9.2.6 Testbench Updates

The existing fetch-stage verification environment should be extended to validate the new Zicfilp functionality.

The following testbenches are expected to be updated:

Testbench	Purpose
`tb_if_stage/tb_if_stage.sv` (IF1)	Verify propagation of CFI metadata through the fetch pipeline.
`tb_if_stage/tb_if_stage.sv` (IF2)	Validate forwarding of LPAD-related instruction packets to the decode stage.
`tb_branch_predictor/tb_module.sv`	Confirm branch prediction behavior remains unchanged after Zicfilp integration.
`tb_top/tb_module.sv`	Execute end-to-end fetch scenarios involving indirect branches and LPAD instructions.

Additional directed test cases should include:

- Valid indirect branch targeting a correct LPAD instruction.
- Indirect branch targeting a non-LPAD instruction.
- Multiple consecutive indirect branches.
- Speculative fetch followed by pipeline flush.
- Branch misprediction together with LPAD verification.
- Recovery after a landing-pad verification failure.

9.3 Instruction Decode Integration (Phase 5B)

9.3.1 Existing Decode Architecture

The Instruction Decode (ID) stage receives instructions from the fetch pipeline and decodes their opcode, funct fields, register operands, immediate values, and execution control signals. The decoder determines the instruction type and generates the control information required by later pipeline stages.

For the existing Sargantana implementation, the decoder already supports integer, floating-point, vector, CSR, branch, load/store, and system instructions. After decoding, all required control information is forwarded toward the Rename stage.

Since the Zicfilp extension introduces a new **Landing Pad (LPAD)** instruction, the decoder must be extended to recognize this instruction and generate additional CFI-related control information.

9.3.2 RTL Files to be Modified

RTL File	Purpose of Modification
`rtl/datapath/id_stage/rtl/decoder.sv`	Decode the LPAD instruction and generate LPAD control signals.
`rtl/control_unit/rtl/control_unit.sv`	Accept new decode signals and distribute them through the pipeline control path if required.
`rtl/datapath/rtl/datapath.sv`	Carry newly generated LPAD metadata toward later pipeline stages.

9.3.3 Required Architectural Changes

The decoder must identify every valid LPAD instruction using its opcode and funct fields defined by the Zicfilp ISA specification. Once detected, the instruction is classified as a CFI instruction rather than a normal arithmetic or branch instruction.

Additional decode outputs should be generated, including:

- LPAD instruction valid signal.
- Decoded LPAD immediate/landing-pad identifier.
- CFI instruction classification bit.
- Exception indication for illegal LPAD encodings.

These signals are forwarded together with the normal decode information so that later stages can perform landing pad verification during indirect control transfers.

9.3.4 New Hardware Components

The decode stage requires only lightweight additions.

New logic includes:

- LPAD instruction decoder.
- LPAD immediate extraction logic.
- CFI control signal generator.
- Additional pipeline control bits for LPAD propagation.

No new memories, queues, or complex execution hardware are introduced at this stage.

9.3.5 Components Remaining Unchanged

The following components continue operating without modification:

- Immediate Generator (`immediate.sv`)
- Vector Set Module (`vset_module.sv`)
- Vector Set Queue (`vset_queue.sv`)
- Existing integer, floating-point, vector, CSR, and memory instruction decoding logic.
- Decode pipeline timing and issue mechanism.

9.3.6 Testbench Updates

The decoder testbench should be extended to verify the new decoding functionality.

New test cases include:

- Correct decoding of valid LPAD instructions.
- Detection of invalid LPAD encodings.
- Verification of generated CFI control signals.
- Mixed instruction streams containing both LPAD and normal instructions.
- Regression testing to ensure existing instruction decoding remains unaffected.

9.4 Rename Stage Integration (Phase 5C)

9.4.1 Existing Rename Architecture

The Rename stage eliminates false data dependencies by mapping architectural registers to physical registers. It manages register allocation through the Rename Table and Free List while dispatching renamed instructions to the Instruction Queue.

Since register renaming is independent of control-flow verification, this stage does not execute any CFI operations. Its primary responsibility during Zicfilp integration is to preserve and forward the CFI metadata generated during instruction decoding.

9.4.2 RTL Files to be Modified

RTL File	Purpose of Modification
`rtl/datapath/ir_stage/rtl/rename_table.sv`	Store and forward LPAD-related metadata with renamed instructions.
`rtl/datapath/ir_stage/rtl/instruction_queue.sv`	Preserve CFI control information while instructions wait for issue.

`rtl/datapath/rtl/datapath.sv`	Extend pipeline buses with LPAD metadata between Decode, Rename, and Register Read stages.

Files remaining unchanged:

- `free_list.sv` (physical register allocation remains identical)

9.4.3 Required Architectural Changes

Rather than performing any landing pad verification, the Rename stage simply propagates the metadata generated during decoding.

Required changes include:

- Extend rename pipeline entries with LPAD control bits.
- Preserve LPAD identifiers during register renaming.
- Store CFI metadata inside the instruction queue.
- Ensure renamed CFI instructions retain all verification information until execution.

No modification is required to the physical register allocation algorithm.

9.4.4 New Hardware Components

Only small metadata fields are added to existing pipeline structures.

These include:

- LPAD metadata fields in rename pipeline registers.
- CFI control bits within Instruction Queue entries.
- Extended datapath signals carrying LPAD information.

No new rename tables, register files, or allocation hardware are required.

9.4.5 Components Remaining Unchanged

The following hardware operates exactly as before:

- Free List (`free_list.sv`)
- Physical register allocation.
- Register renaming algorithm.
- Dependency tracking.
- Register reclamation logic.
- Issue scheduling mechanism.

9.4.6 Testbench Updates

The Rename stage verification environment should include new tests to verify correct propagation of CFI metadata.

Additional tests include:

- LPAD instruction successfully renamed and forwarded.
- Mixed CFI and non-CFI instruction sequences.
- Correct storage of LPAD metadata inside the Instruction Queue.
- Verification that physical register allocation is unaffected.
- Regression testing to ensure existing rename functionality remains unchanged.

9.5 Register Read Stage Integration (Phase 5D)

9.5.1 Existing Register Read Architecture

The Register Read (RR) stage reads the source operands from the physical register file after register renaming. It supplies operand values to the execution stage and performs no instruction decoding or control-flow validation. Since the Zicfilp extension validates landing pads using the target instruction rather than register contents, only pipeline metadata propagation is required.

9.5.2 RTL Files to be Modified

Primary RTL Modifications

RTL File	Purpose
`rtl/datapath/rtl/rr_stage/rtl/regfile.sv`	Forward LPAD-related metadata alongside register operands.
`rtl/datapath/rtl/datapath.sv`	Carry new LPAD pipeline signals from RR to Execute.

Supporting RTL Changes (Interface / Metadata Propagation)

RTL File	Purpose
`rtl/datapath/rtl/ir_stage/rtl/instruction_queue.sv`	Preserve decoded LPAD information until RR stage.
`rtl/control_unit/rtl/control_unit.sv`	Extend pipeline control signals if additional metadata bits are introduced.

9.5.3 Required Architectural Changes

- Introduce LPAD metadata into the RR pipeline register.
- Forward decoded landing-pad information to Execute.
- Preserve existing register read latency.
- Ensure operand forwarding remains unaffected.
- Keep hazard detection identical to the original processor.

9.5.4 New Hardware Components

No new functional hardware is introduced.

Only lightweight pipeline signals are added:

- LPAD Valid bit
- LPAD Metadata bus
- Pipeline metadata register

9.5.5 Components Remaining Unchanged

The following modules continue operating without modification:

- Register File
- Read Ports
- Operand Forwarding Network
- Physical Register Storage
- Register Rename Mechanism
- Dependency Resolution Logic

9.5.6 Testbench Updates

The RR testbench should verify:

- Correct propagation of LPAD metadata.
- Normal register reads remain unchanged.
- Hazard detection behaves correctly.
- No additional pipeline latency.

9.6 Execution Stage Integration (Phase 5E)

9.6.1 Existing Execution Architecture

The Execute stage performs arithmetic, branch resolution, memory address generation, multiplication, division, floating-point execution, and SIMD processing. Since indirect CALL/JALR instructions are resolved here, this stage is responsible for enforcing the Zicfilp landing-pad verification before control is transferred.

9.6.2 RTL Files to be Modified

Primary RTL Modifications

RTL File	Purpose
`rtl/datapath/rtl/exe_stage/rtl/exe_stage.sv`	Integrate the Landing Pad verification pipeline.
`rtl/datapath/rtl/exe_stage/rtl/branch_unit.sv`	Validate indirect CALL/JALR targets before branch completion.
`rtl/datapath/rtl/datapath.sv`	Connect LPAD signals, exceptions and pipeline control.

Supporting RTL Changes (Interface / Metadata Propagation)

RTL File	Purpose
`rtl/control_unit/rtl/control_unit.sv`	Generate pipeline flush and software-check exception control.
`rtl/datapath/rtl/exe_stage/rtl/score_board_scalar.sv`	Stall instruction retirement until LPAD verification completes (if required).
`rtl/datapath/rtl/exe_stage/rtl/pending_mem_req_queue.sv`	Track outstanding instruction fetch requests if LPAD verification requires additional memory access.
`rtl/datapath/rtl/exe_stage/rtl/mem_unit.sv`	Interface updates only if instruction memory access path is extended.

Expected to Remain Unchanged

Integer ALU

- alu.sv
- alu_add.sv
- alu_cmp.sv
- alu_logic.sv
- alu_shift.sv
- alu_count_pop.sv
- zero_counter/*

Integer Arithmetic

- mul_unit.sv
- div_unit.sv
- div_4bits.sv

Floating Point

- Entire `fpu/`
- Entire `old_fpu/`

SIMD

- Entire `simd/`

Memory System

- load_store_queue.sv
- store_buffer.sv
- vagu.sv

These modules perform computation only and are independent of landing-pad verification.

9.6.3 Required Architectural Changes

The Execute stage is extended to enforce Zicfilp by:

- Detecting indirect CALL/JALR instructions.
- Receiving LPAD metadata from earlier pipeline stages.
- Verifying that the target instruction is a valid LPAD.
- Comparing the expected landing-pad information with the fetched instruction.
- Allowing execution only after successful verification.
- Raising a Software Check Exception on failure.
- Initiating pipeline flush and recovery when verification fails.

9.6.4 New Hardware Components

Hardware Component	Purpose
Landing Pad Checker	Detect LPAD instruction at the branch target.
LPAD Metadata Comparator	Compare decoded LPAD metadata with expected values.
CFI Verification Logic	Determine whether indirect control transfer is legal.

Software Check Exception Generator	Raise exception upon verification failure.
Pipeline Flush Controller	Remove speculative instructions after a failed verification.

9.6.5 Components Remaining Unchanged

- Integer ALU
- Multiplier
- Divider
- Floating Point Unit
- SIMD Execution Units
- Load Store Queue
- Store Buffer
- Memory Data Path
- Vector Address Generation Unit
- Existing arithmetic datapaths

9.6.6 Testbench Updates

The Execute stage testbench should be extended with:

- Valid indirect CALL → LPAD.
- Valid indirect JALR → LPAD.
- Invalid target without LPAD.
- Incorrect LPAD metadata.
- Software-check exception generation.
- Pipeline flush verification.
- Regression tests confirming ALU, MUL, DIV, FPU and SIMD functionality remain unaffected.

9.7 Writeback and Graduation (Commit) Integration (Phase 5F)

9.7.1 Existing Commit Architecture

After execution completes, results are propagated through the backend before finally committing architectural state.

The current backend consists of:

- **Execution Stage (`exe_stage`)**
  - Produces ALU, multiplier, divider, floating-point, SIMD and memory results.
  - Maintains scoreboards and pending operation queues.
- **Register Read / Register File (`rr_stage`)**
  - Provides architectural register storage.
  - Receives committed writeback values.
- **Writeback Stage (`wb_stage`)**
  - Uses the Graduation List (ROB-like structure).
  - Commits instructions in program order.
  - Updates architectural register mappings.
  - Retires completed instructions.

The existing pipeline assumes every instruction reaching graduation is architecturally valid.

For CFI, this assumption is no longer true.

Before a JALR instruction commits, the processor must verify that execution arrived at a legal landing pad. If verification fails, the instruction must never update architectural state.

9.7.2 RTL Files to be Modified

Unlike previous phases, the commit stage requires modifications across several pipeline components because the CFI status generated earlier must propagate until retirement.

Primary RTL modifications

➢ rtl/datapath/rtl/wb_stage/rtl/graduation_list.sv

Add:

- CFI valid bit
- CFI exception bit
- landing-pad verification result
- shadow stack status
- commit gating logic

➢ rtl/datapath/rtl/datapath.sv

Update pipeline interfaces.

Propagate

- cfi_valid
- cfi_exception

- lpad_verified
- shadow_stack_ok

from execution stage into writeback.

➢ rtl/datapath/rtl/exe_stage/rtl/exe_stage.sv

Extend outputs to generate

- landing pad verification result
- shadow stack verification status
- CFI trap request

➢ rtl/control_unit/rtl/control_unit.sv

Handle

- CFI exceptions
- pipeline flush
- redirect PC
- trap generation

➢ rtl/datapath/rtl/rr_stage/rtl/regfile.sv

No functional redesign.

Only ensure writes are blocked whenever commit is cancelled because of a CFI violation.

➢ rtl/datapath/rtl/interface_csr/rtl/csr_interface.sv

Update CSR exception interface.

Allow CFI traps to be reported through the standard exception mechanism.

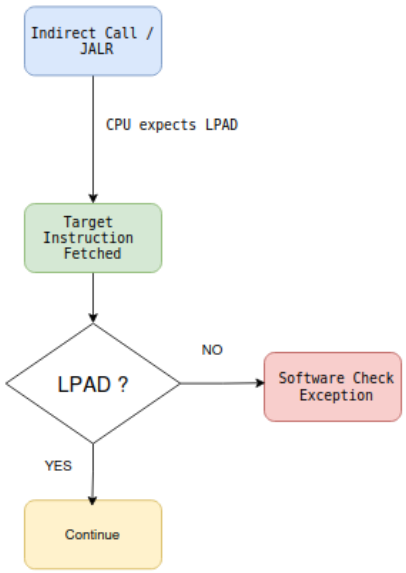
➢ Possible integration may also require small interface updates inside
rtl/datapath/rtl/ir_stage/rtl/instruction_queue.sv
rtl/datapath/rtl/ir_stage/rtl/rename_table.sv
rtl/datapath/rtl/ir_stage/rtl/free_list.sv

to correctly squash speculative instructions following a CFI violation.

9.7.3 Required Architectural Changes

The commit stage must become CFI-aware.

Instead of retiring every completed instruction, retirement proceeds as follows.



The graduation logic must therefore

- inspect CFI verification status,
- prevent architectural register updates on failure,
- flush younger speculative instructions,
- redirect control to the exception handler,
- maintain precise exceptions.

For Shadow Stack support, return instructions must also verify shadow stack integrity before retirement.

9.7.4 New Hardware Components

Several lightweight control structures are added around the existing graduation logic.

1. CFI Commit Checker

Final verification before retirement.

Inputs

- Landing pad status
- Shadow stack status
- Exception bits

Outputs

- commit_enable
- cfi_trap

2. Commit Control Logic

Generates

- pipeline flush
- rollback request
- retirement enable

3. CFI Status Bits

Additional metadata stored with each instruction.

Example fields

cfi_valid

landing_pad_verified

shadow_stack_verified

cfi_exception

4. Trap Interface

Connects

- Commit Stage
- CSR Interface
- Exception Handler

9.7.5 Components Remaining Unchanged

Most of the processor remains unchanged.

The following RTL blocks continue operating without architectural modification.

Control blocks

rtl/common_cells/

Decode stage

rtl/datapath/rtl/id_stage/

Modules

- decoder.sv
- immediate.sv
- vset_module.sv
- vset_queue.sv

Fetch prediction hardware

rtl/datapath/rtl/if_stage_1/

- bimodal_predictor.sv
- branch_predictor.sv
- return_address_stack.sv

rtl/datapath/rtl/if_stage_2/

- if_stage_2.sv

Execution functional units

No computational changes are required inside

rtl/datapath/rtl/exe_stage/rtl/alu/

including

- alu_add.sv
- alu_cmp.sv
- alu_logic.sv
- alu_shift.sv
- alu_count_pop.sv
- zero_counter/*

Likewise these remain functionally unchanged

- mul_unit.sv
- div_unit.sv
- div_4bits.sv
- mem_unit.sv
- load_store_queue.sv
- pending_mem_req_queue.sv
- pending_fp_ops_queue.sv
- score_board_scalar.sv
- score_board_simd.sv
- store_buffer.sv
- vagu.sv

Floating-point hardware

Entire directories remain unchanged

rtl/datapath/rtl/exe_stage/rtl/fpu/
rtl/datapath/rtl/exe_stage/rtl/old_fpu/

rtl/datapath/rtl/exe_stage/rtl/simd/

because CFI only validates control flow and does not alter arithmetic execution.

9.7.6 Testbench Updates

Verification must now include commit-stage validation.

Update existing writeback tests

rtl/datapath/rtl/wb_stage/tb/

especially

tb_graduation_list/

- ➢ Add tests for
  - successful retirement after valid LPAD
  - blocked retirement after invalid LPAD
  - shadow stack mismatch
  - precise exception generation
  - pipeline flush after CFI violation
  - no architectural register corruption

Update datapath integration tests

rtl/datapath/rtl/exe_stage/tb/tb_top/

Verify complete pipeline behaviour

- ❖ Fetch
- ❖ Decode
- ❖ Rename
- ❖ Execute
- ❖ Landing Pad Check
- ❖ Commit

Update control unit testbench

rtl/control_unit/tb/

Verify

- CFI exception generation
- trap request
- flush control
- pipeline recovery

9.8 Phase Completion Summary

At the end of Phase 9:

- ✓ Landing-pad verification reaches the retirement stage.
- ✓ Shadow-stack status is checked before architectural commit.
- ✓ `graduation_list.sv` becomes CFI-aware.
- ✓ `exe_stage.sv` exports CFI verification results to the backend.
- ✓ `datapath.sv` propagates CFI metadata across pipeline stages.
- ✓ `control_unit.sv` handles CFI trap generation and pipeline recovery.
- ✓ `csr_interface.sv` forwards CFI exceptions through the existing CSR mechanism.
- ✓ `regfile.sv` only commits results from CFI-approved instructions.
- ✓ Existing execution units (ALU, multiplier, divider, memory, FPU, SIMD, scoreboards, queues, predictors) remain functionally unchanged, requiring only interface connectivity where applicable.
- ✓ Writeback, graduation, datapath, and control-unit testbenches are extended with CFI-specific retirement, trap, and pipeline flush test cases.

Chapter 10

Phase 6 – Zicfilp Verification and Validation

10.1 Verification Objectives

- Verify correct implementation of the Zicfilp (Landing Pad) extension.
- Ensure valid indirect control transfers execute only through authorized Landing Pads.
- Validate correct detection and handling of invalid control-flow transfers.
- Verify seamless integration with the existing Sargantana pipeline without affecting non-CFI instructions.

10.2 RTL Simulation

Verify all modified RTL modules using SystemVerilog testbenches.

Simulate the processor using Verilator or QuestaSim to ensure:

- Correct Landing Pad detection.
- Proper execution of LPAD instructions.
- Correct propagation of CFI metadata through the pipeline.
- Proper exception generation for CFI violations.

10.3 Directed Tests

Execute hand-written RISC-V assembly programs to validate Landing Pad behavior.

Test cases include:

- Valid indirect function calls.
- Valid indirect jumps.
- Correct execution of LPAD instructions.
- Invalid indirect jumps targeting non-LPAD instructions.
- Exception generation after Landing Pad verification failure.
- Pipeline recovery following CFI exceptions.

10.4 ISA Compliance Tests

Execute official RISC-V CFI compliance tests for the Zicfilp extension.

Verify:

- Architectural correctness.
- ISA specification compliance.
- Proper handling of legal and illegal indirect control transfers.
- Correct exception behavior as defined by the RISC-V CFI specification.

10.5 Linux / Emulator Validation

Run software workloads on the Sargantana emulator after integrating the Zicfilp extension.

Validate:

- Correct execution of normal applications.
- Successful handling of indirect branches and function calls.
- Proper operation of Landing Pad verification during realistic workloads.
- No regression in existing processor functionality.

10.6 Performance Evaluation

Measure the hardware and performance impact introduced by the Landing Pad mechanism.

Evaluate:

- Additional pipeline latency.
- Branch resolution overhead.
- Impact on instruction throughput.
- Area and control logic overhead.
- Overall execution performance compared with the baseline processor.

10.7 Expected Outcome

The implementation should provide:

- A functionally correct Zicfilp implementation.
- Accurate Landing Pad verification for indirect control-flow transfers.
- Correct exception generation for invalid control-flow targets.
- Full compliance with the RISC-V Zicfilp specification.
- Successful validation across RTL simulation, ISA compliance testing, and system-level execution.

10.8 Summary

Verification Type	What is Tested	How it is Performed	Goal

RTL Simulation	Landing Pad verification logic and pipeline integration	SystemVerilog testbenches (Verilator/QuestaSim)	Verify RTL functionality
Directed Tests	Valid and invalid indirect control-flow transfers	Hand-written RISC-V assembly programs	Verify correct Landing Pad behavior
ISA Compliance Tests	Zicfilp ISA correctness	Official RISC-V CFI compliance test suite	Ensure specification compliance
Linux / Emulator Tests	Complete processor execution	Run software on the Sargantana emulator	Validate real-world processor behavior
Performance Evaluation	Pipeline and hardware overhead	Compare execution before and after Zicfilp integration	Measure security overhead and performance impact

Chapter 11

Project Timeline

11.1 Weekly Schedule

Week	Date	Planned Activities
Week 1	**September 1 – September 7**	**Preparation and Project Onboarding** – Community onboarding meetings, mentor discussions, repository setup, development environment verification, review of coding standards, project planning, and creation of initial GitHub issues.
Week 2	**September 8 – September 14**	**Phase 1 – Zicfiss Architecture Analysis and Design**
Week 3	**September 15 – September 21**	**Phase 2A & Phase 2B** – Initial Zicfiss RTL implementation and pipeline integration, alongside **RTL simulation** and functional verification from Phase 3.
Week 4	**September 22 – September 28**	**Phase 2C & Phase 2D** – Continue RTL implementation, integrate remaining pipeline components, and perform continuous RTL simulation and debugging.
Week 5	**September 29 – October 5**	**Phase 2E & Phase 2F** – Complete RTL implementation, finalize Shadow Stack integration, and complete RTL simulation and functional verification.
Week 6	**October 6 – October 12**	**Phase 3 – System-Level Verification** – Directed assembly tests, RISC-V Architectural Compliance Tests (ACT), Linux/Emulator validation, performance evaluation, and hardware overhead analysis.
Week 7	**October 13 – October 19**	**Phase 4 – Zicfilp Architecture Analysis and Design**
Week 8	**October 20 – October 26**	**Phase 5A & Phase 5B** – Initial Zicfilp RTL implementation and Landing Pad integration, alongside **RTL simulation** and functional verification from Phase 6.

Week 9	**October 27 – November 2**	**Phase 5C & Phase 5D** – Continue Zicfilp RTL implementation, pipeline integration, and ongoing RTL simulation and debugging.
Week 10	**November 3 – November 9**	**Phase 5E & Phase 5F** – Complete Landing Pad RTL implementation and finalize RTL simulation and functional verification.
Week 11	**November 10 – November 16**	**Phase 6 – System-Level Verification** – Directed assembly tests, RISC-V Architectural Compliance Tests (ACT), Linux/Emulator validation, performance evaluation, and final validation of the Zicfilp implementation.
Week 12	**November 17 – November 23**	**Complete Processor Validation** – End-to-end testing, integration testing of both Zicfiss and Zicfilp, debugging, optimization, optional synthesis (if required), RTL cleanup, and resolution of mentor review comments.
Week 13	**November 24 – November 30**	**Documentation and Final Submission** – Complete technical documentation, update architecture diagrams, prepare user/developer documentation, submit remaining Pull Requests, resolve final review comments, and prepare the final implementation for upstream integration into the Sargantana repository.

11.2 Planned GitHub Issues (to be opened during mentorship)

The project will follow an issue-driven development workflow, where each major milestone is tracked through dedicated GitHub Issues before implementation begins.

Approximately **17 GitHub Issues** are expected throughout the mentorship:

Issue Category	Approx. Issues
Phase 1 – Zicfiss Architecture & Design	1
Phase 2A–2F – Zicfiss RTL Implementation	6
Phase 3 – Zicfiss Verification & Validation	1
Phase 4 – Zicfilp Architecture & Design	1
Phase 5A–5F – Zicfilp RTL Implementation	6

Phase 6 – Zicfilp Verification & Validation	1
Documentation & Final Integration	1
Total	**≈17 Issues**

The exact number of issues may vary depending on mentor feedback and project evolution during the mentorship.

11.3 Planned Pull Requests

Rather than submitting a single large contribution, development will be carried out through multiple incremental Pull Requests.

Expected Pull Requests include:

- Architecture and design updates
- Zicfiss RTL implementation (multiple PRs)
- Zicfiss verification and validation
- Zicfilp RTL implementation (multiple PRs)
- Zicfilp verification and validation
- Documentation improvements
- Final integration and cleanup

Approximately **15–25 Pull Requests** are expected during the mentorship, depending on review feedback and how individual features are split into independent contributions.

Each Pull Request will undergo mentor review before merging, enabling continuous feedback, easier code review, and smoother upstream integration into the Sargantana repository.